Adaptive Terminal-Based Typing Tutor with Real-Time Telemetry Simulation

Karthik Reddy
Enrollment ID: 230035

Nilesh Chakrabarty
Enrollment ID: 230053

Nikhil Raj
Enrollment ID: 230080

***Abstract*—This paper presents the design and implementation of the KNN Typing Tutor, a highly optimized, terminal-based touch-typing application written in pure C. The acronym "KNN" represents the collaborative effort of the authors (Karthik, Nilesh, Nikhil) rather than the K-Nearest Neighbors algorithm. The system features a zero-dependency architecture utilizing a custom memory allocator, POSIX termios for non-blocking raw input, and ANSI escape sequences for a high-performance, flicker-free command-line interface. To augment the terminal experience, a web-based telemetry simulation was developed using Node.js and Next.js, allowing for real-time remote tracking of keystroke dynamics and ghost replays. The engine calculates typing proficiency using strict industry-standard Net Words Per Minute (WPM) formulas, and incorporates an adaptive algorithm that dynamically constructs practice texts heavily weighted toward the user's statistically weakest keystrokes.**

***Index Terms*—Touch Typing, Command Line Interface, ANSI Escape Sequences, Telemetry, Keystroke Dynamics, C Programming**

I. Introduction

In the modern software engineering landscape, the Command Line Interface (CLI) remains a ubiquitous tool. Despite the prevalence of web-based typing tutors such as Monkeytype or Typing.com, there is a distinct lack of premium, lightweight typing environments that execute natively within a developer's terminal. Web-based solutions introduce inherent browser latency, which can skew the microsecond-level precision required to accurately measure keystroke dynamics.

To bridge this gap, we developed the KNN Typing Tutor. The core objective is to provide developers with a zero-latency, highly accurate typing tutor that runs natively in UNIX-based terminals without external dependencies. Furthermore, the system is designed to provide deep analytical feedback—such as per-finger speed analysis and keystroke latency heatmaps—while utilizing a web-based telemetry simulation backend to visualize data externally [3].

II. Literature Review

The domain of touch-typing education has evolved significantly from early desktop applications like Mavis Beacon Teaches Typing to modern web applications.

- **GNU Typist (gtypist):** A traditional terminal-based typing tutor. While highly functional, it relies heavily on the ncurses library, which introduces rendering overhead and dictates the UI paradigm. It lacks modern analytics such as keystroke-level latency tracking.
- **Monkeytype:** A popular open-source web application that offers highly customizable typing tests. While it provides excellent analytics, its execution within a browser sandbox relies on JavaScript event listeners, which are subject to garbage collection pauses and browser-level event loop delays.
- **Keystroke Dynamics Literature:** Research by Monrose and Rubin (1997) highlights that measuring the exact dwell and flight times between keystrokes can uniquely identify users and highlight cognitive or physical bottlenecks.

The KNN Typing Tutor improves upon existing CLI tutors by stripping away heavy UI libraries like ncurses in favor of direct ANSI escape sequence manipulation, ensuring zero-overhead rendering while providing the advanced analytical depth typically reserved for heavy web applications [1].

III. System Architecture

The system is fundamentally bifurcated into a high-performance Terminal Client (Core) and a Web-based Telemetry Simulation.

A. Core Engine (C Terminal Client)

The core application is written in standard C [1], focusing on extreme performance and minimal footprint [2].

- **Raw Mode Input Handling:** The POSIX `termios.h` library [3] is utilized to disable canonical mode and terminal echo. This allows the application to read keystrokes instantaneously as bytes arrive in the stdin buffer, bypassing the line-buffer entirely.
- **ANSI Rendering Engine:** Instead of relying on ncurses, the UI is rendered using raw ANSI escape codes. To prevent terminal scrollback pollution (a common issue in CLI applications), the application utilizes the `\033[H\033[J` sequence to reset the cursor to the viewport origin and clear downwards, achieving flicker-free 60+ FPS in-place updates.
- **Custom Memory Allocator:** To prevent fragmentation during long sessions, the system implements a custom 64 KB bump allocator (`my_alloc`). Memory is linearly assigned and instantaneously reset at the beginning of each session via `reset_memory()`, eliminating the overhead of standard `malloc`/`free` cycles [2].

B. Web Telemetry Simulation

While the primary application runs in the terminal, a secondary subsystem was built to simulate remote telemetry and data aggregation.

- **Backend (Node.js/Express):** A lightweight REST API that receives highly detailed session payloads, including timestamp arrays and individual keystroke boolean flags. It persists these "attempts" in a JSON-based datastore.
- **Frontend (Next.js):** A modern React application utilizing TailwindCSS and Framer Motion. It fetches the terminal's telemetry data to render visual heatmaps and simulate "Ghost" replays of previous typing sessions.

IV. Methodology

A. Industry-Standard Net WPM Calculation

Typing speed is calculated using the strict industry-standard formula, which penalizes uncorrected errors heavily. A "Word" is strictly defined as 5 characters, including spaces.

$$\text{Gross WPM} = \frac{\text{Total Characters Typed}/5}{\text{Time (minutes)}} \tag{1}$$

$$\text{Net WPM} = \text{Gross WPM} - \frac{\text{Uncorrected Errors}}{\text{Time (minutes)}} \tag{2}$$

In the C implementation, to avoid floating-point inaccuracies, the formula is algebraically simplified to integer mathematics: `(correct_chars * 12) / elapsed_seconds`.

B. Adaptive Difficulty Algorithm

The application includes a dynamic "Adaptive Mode". During normal sessions, the analytics engine records the latency of every individual character and tracks error frequencies. The `get_slowest_keys()` function aggregates this data to identify the user's "weakest" keys. When Adaptive Mode is engaged, the text generation algorithm dynamically constructs a custom string by probabilistically sampling words from the dictionary that contain the user's weakest keys (e.g., a 70% bias towards words containing the weak key, and 30% random).

C. Edge Case Mitigation

Extensive logic was implemented to handle edge cases common in terminal applications:

- **Buffer Overflow Prevention:** Input duration is capped, restricting the generated target strings to a strict maximum of 500 words to ensure the 8192-byte stack buffers are never overrun.
- **Input Buffer Flushing:** If a user continues to type after a timed session concludes, the stdin buffer is rapidly flushed using a non-blocking read loop (`enable_raw_mode_nb()`) before transitioning to the analytics screen. This prevents accidental keystrokes from prematurely dismissing the UI.
- **Session Abort Integrity:** Pressing ESC mid-session instantly flags the state as aborted, preventing partial, low-accuracy data from permanently corrupting the user's historical profile and personal bests.

V. Results and Discussion

The resulting application is highly robust. By utilizing direct termios input and a custom memory allocator, the terminal client operates with near-zero latency, correctly logging keystroke intervals down to the millisecond using `gettimeofday`. The analytics dashboard successfully processes these intervals to provide a graphical WPM sparkline graph and a color-coded keyboard heatmap directly in the terminal, utilizing 256-color ANSI codes.

The web telemetry simulation successfully proves the extensibility of the CLI application, allowing local terminal data to be offloaded and analyzed in a modern browser environment without compromising the native terminal typing experience.

VI. Future Scope

Several directions exist for extending the KNN Typing Tutor beyond its current implementation:

- **Machine Learning-Based Adaptation:** The current adaptive algorithm uses a fixed probabilistic bias. A future iteration could employ a lightweight on-device model (e.g., a recurrent neural network) trained on keystroke latency sequences to predict and proactively target emerging weak keys before errors manifest.
- **Multi-Platform Support:** The present implementation is constrained to UNIX-based systems due to its reliance on POSIX `termios` and ANSI sequences. Porting raw-mode input handling to Windows via the Win32 Console API would broaden accessibility significantly.
- **Persistent Cloud Telemetry:** The current web telemetry subsystem uses a local JSON datastore. Replacing this with a cloud-backed database (e.g., PostgreSQL or Firebase) would enable cross-device session history, long-term progress tracking, and community leaderboards.
- **Language and Script Support:** Currently, the tutor is limited to ASCII-range characters. Adding support for Unicode and multi-byte input sequences would enable practice in non-Latin scripts such as Devanagari or CJK character sets.
- **Biometric Authentication via Keystroke Dynamics:** Building on the foundational work of Monrose and Rubin, the recorded dwell and flight-time profiles could be used to authenticate users passively, removing the need for password-based login to the telemetry dashboard.

VII. Conclusion

The KNN Typing Tutor successfully demonstrates that advanced, highly analytical touch-typing software does not require heavy browser environments or UI frameworks. By leveraging fundamental UNIX concepts [3], ANSI escape sequences, and rigorous mathematical standards for performance evaluation, the application provides a premium, zero-latency

experience directly where developers spend most of their time: the terminal.

ACKNOWLEDGMENT

The authors thank their institution for providing the resources and environment necessary to develop and test this project.

REFERENCES

[1] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Englewood Cliffs, NJ, USA: Prentice Hall, 1988.

[2] R. E. Bryant and D. R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed. Pearson, 2015.

[3] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Addison-Wesley Professional, 2013.

[4] F. Monrose and A. D. Rubin, "Authentication via keystroke dynamics," in *Proc. 4th ACM Conf. Comput. Commun. Security*, Zurich, Switzerland, 1997, pp. 48–56.